

Name - Vidya Dharne , Roll no- cse12

Aim: To implement a Simple Linear Regression model using Python and predict MPG from Horsepower using OLS.

Objectives

- Understand SLR
- Load/preprocess dataset
- Perform EDA
- Compute OLS coefficients
- Evaluate with SSE, MSE, R^2
- Visualize regression and residual plots

Software Requirements

- Python 3.x
- Jupyter/Colab/VS Code
- Libraries: pandas, numpy, matplotlib, seaborn

Theory:

Linear regression models the relationship between a target continuous variable Y and an input feature vector X using a linear equation:

$$\hat{y} = w_0 + w_1 x_1$$

In matrix form, including the bias/intercept term $X_0=1$:

$$\hat{y} = Xw$$

To find the optimal weight vector w^* , the Ordinary Least Squares (OLS) method minimizes the Sum of Squared Errors (SSE) between actual target values y and predicted values $\hat{y}$:

$$SSE = \sum_{i=1}^N (y_i - \hat{y}_i)^2 = y^T y - X^T w^T y - X^T w$$

Solving for w analytically yields the normal equation:

$$w^* = (X^T X)^{-1} X^T y$$

Note: The inverse $(X^T X)^{-1}$ exists only if the matrix $X^T X$ is non-singular (i.e., its determinant is non-zero).

Algorithm:

```
In [ ]: #1.Import libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

```
In [ ]: #2.Load Auto MPG dataset
# Auto MPG dataset from UCI repository
url = "https://archive.ics.uci.edu/ml/machine-learning-databases/auto-mpg/auto-mpg.data"

column_names = ['mpg', 'cylinders', 'displacement', 'horsepower', 'weight',
                 'acceleration', 'model_year', 'origin', 'car_name']

df = pd.read_csv(url, names=column_names, delim_whitespace=True, na_values='?')
df.head()
```

```
/tmp/ipykernel_1603/1802870797.py:8: FutureWarning: The 'delim_whitespace' keyword in pd.read_csv is deprecated and will be removed in a future version. Use ``sep='\s+'`` instead
df = pd.read_csv(url, names=column_names, delim_whitespace=True, na_values='?')
```

```
Out [ ]:
```

	mpg	cylinders	displacement	horsepower	weight	acceleration	model_year	origin	car_name
0	18.0	8	307.0	130.0	3504.0	12.0	70	1	chevrolet chevelle malibu
1	15.0	8	350.0	165.0	3693.0	11.5	70	1	buick skylark 320
2	18.0	8	318.0	150.0	3436.0	11.0	70	1	plymouth satellite
3	16.0	8	304.0	150.0	3433.0	12.0	70	1	amc rebel sst
4	17.0	8	302.0	140.0	3449.0	10.5	70	1	ford torino

```
In [ ]: #3. Inspect and clean data
# Inspect data
print(df.info())
print(df.isnull().sum())

# 'horsepower' has missing values marked as '?' -> already converted to NaN above
# Drop rows with missing horsepower values
df = df.dropna(subset=['horsepower'])

# Ensure horsepower is numeric
df['horsepower'] = df['horsepower'].astype(float)

print("Shape after cleaning:", df.shape)

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 398 entries, 0 to 397
Data columns (total 9 columns):
#   Column          Non-Null Count  Dtype
---  ---
0    mpg             398 non-null    float64
1    cylinders        398 non-null    int64
2    displacement     398 non-null    float64
3    horsepower       392 non-null    float64
4    weight           398 non-null    float64
5    acceleration     398 non-null    float64
6    model_year       398 non-null    int64
7    origin           398 non-null    int64
8    car_name         398 non-null    object
dtypes: float64(5), int64(3), object(1)
memory usage: 28.1+ KB
None
mpg             0
cylinders        0
displacement     0
horsepower       6
weight           0
acceleration     0
model_year       0
origin           0
car_name         0
dtype: int64
Shape after cleaning: (392, 9)
```

```
In [ ]: #4. Select horsepower and mpg

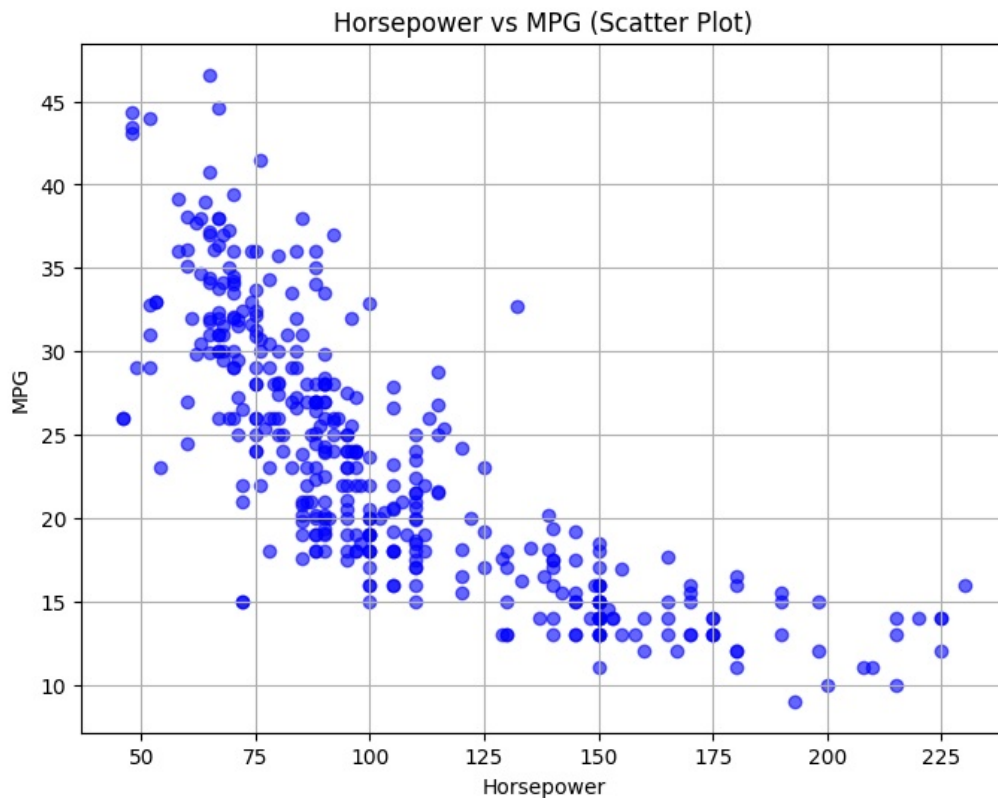
X = df['horsepower'].values # independent variable
y = df['mpg'].values        # dependent variable

print("X shape:", X.shape)
print("y shape:", y.shape)

X shape: (392,)
y shape: (392,)
```

```
In [ ]: #5. Plot scatter graph

plt.figure(figsize=(8,6))
plt.scatter(X, y, color='blue', alpha=0.6)
plt.xlabel('Horsepower')
plt.ylabel('MPG')
plt.title('Horsepower vs MPG (Scatter Plot)')
plt.grid(True)
plt.show()
```



```
In [ ]: #6.Add bias term

# Add a column of ones to X for the bias/intercept term
X_bias = np.c_[np.ones(X.shape[0]), X]
print(X_bias[:5])

[[ 1. 130.]
 [ 1. 165.]
 [ 1. 150.]
 [ 1. 150.]
 [ 1. 140.]]
```

```
In [ ]: #7.Compute correlation

correlation = np.corrcoef(X, y)[0, 1]
print("Correlation between horsepower and mpg:", correlation)

Correlation between horsepower and mpg: -0.7784267838977756
```

```
In [ ]: #8.Calculate OLS weights

# OLS Formula: w = (X^T X)^-1 X^T y
w = np.linalg.inv(X_bias.T @ X_bias) @ X_bias.T @ y

intercept = w[0]
slope = w[1]

print("Intercept (b0):", intercept)
print("Slope (b1):", slope)

Intercept (b0): 39.9358610211705
Slope (b1): -0.15784473335365387
```

```
In [ ]: #9.Predict values

y_pred = X_bias @ w
print(y_pred[:10])

[19.41604569 13.89148002 16.25915102 16.25915102 17.83759835  8.68260382
  5.21001968  5.99924335  4.42079602  9.94536168]
```

```
In [ ]: #10.Compute residuals

residuals = y - y_pred
print(residuals[:10])

[-1.41604569  1.10851998  1.74084898 -0.25915102 -0.83759835  6.31739618
  8.78998032  8.00075665  9.57920398  5.05463832]
```

```
In [ ]: #11. Calculate SSE, MSE, R2
```

```
n = len(y)

# Sum of Squared Errors
SSE = np.sum(residuals**2)

# Mean Squared Error
MSE = SSE / n

# Total Sum of Squares
SST = np.sum((y - np.mean(y))**2)

# R-squared
R2 = 1 - (SSE / SST)

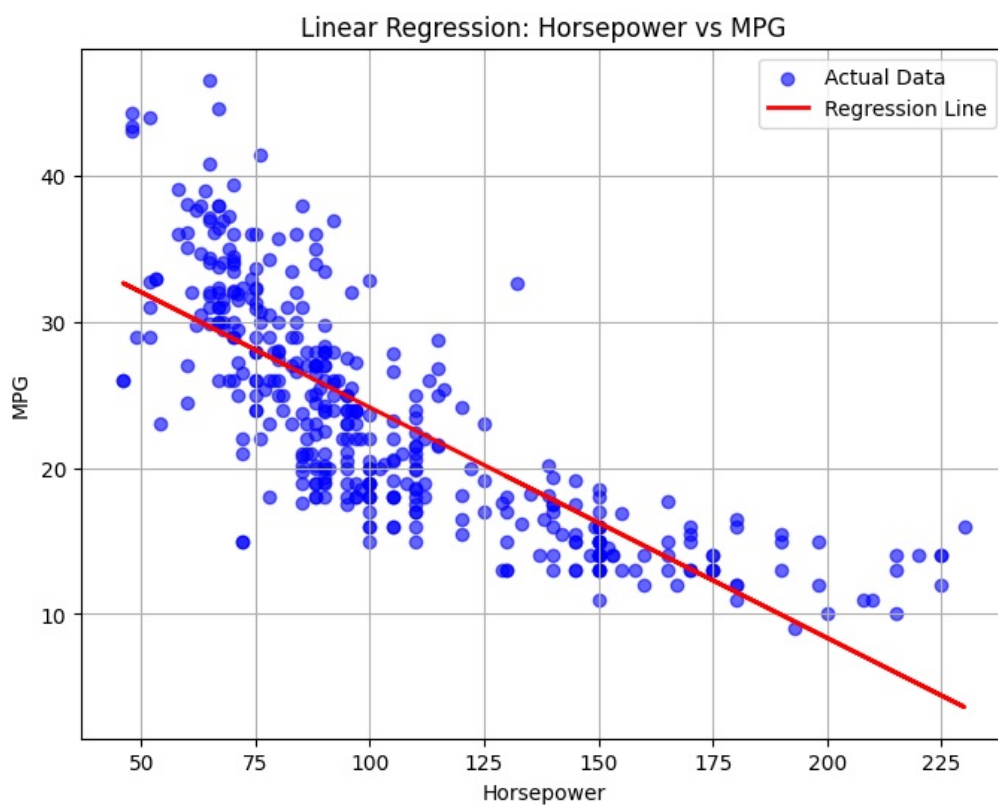
print("SSE:", SSE)
print("MSE:", MSE)
print("R2:", R2)
```

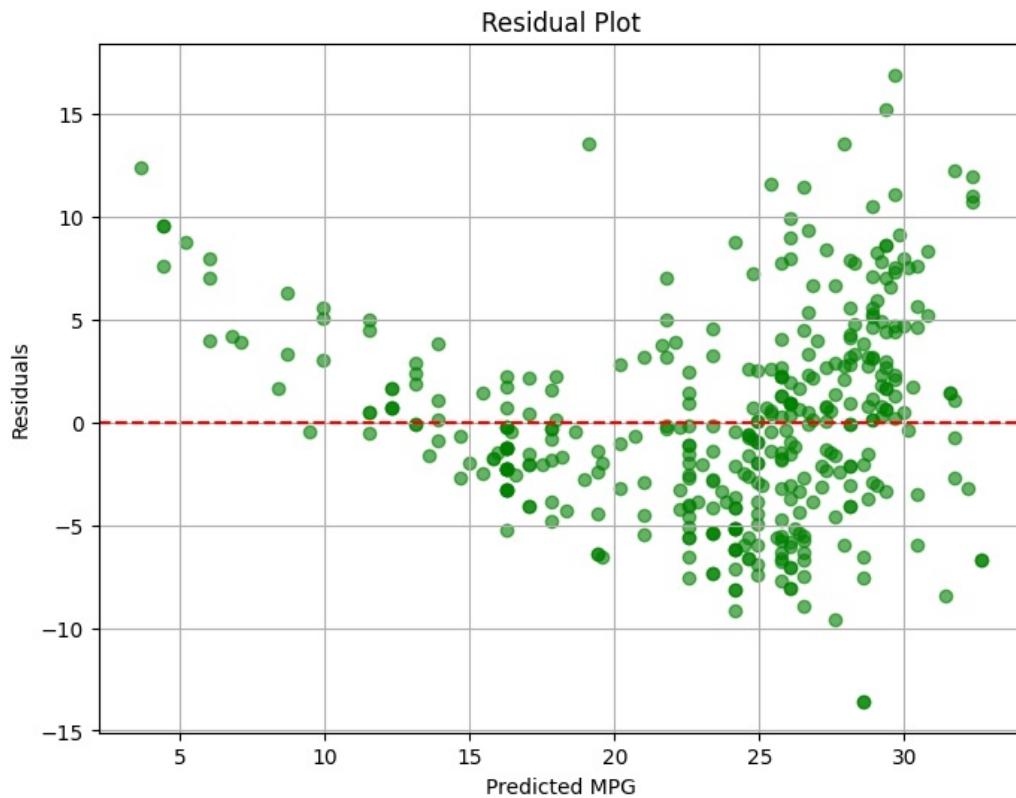
SSE: 9385.915871932419
MSE: 23.943662938603108
R²: 0.6059482578894348

```
In [ ]: #12. Plot regression and residual graphs
```

```
# Regression Line Plot
plt.figure(figsize=(8,6))
plt.scatter(X, y, color='blue', alpha=0.6, label='Actual Data')
plt.plot(X, y_pred, color='red', linewidth=2, label='Regression Line')
plt.xlabel('Horsepower')
plt.ylabel('MPG')
plt.title('Linear Regression: Horsepower vs MPG')
plt.legend()
plt.grid(True)
plt.show()

# Residual Plot
plt.figure(figsize=(8,6))
plt.scatter(y_pred, residuals, color='green', alpha=0.6)
plt.axhline(y=0, color='red', linestyle='--')
plt.xlabel('Predicted MPG')
plt.ylabel('Residuals')
plt.title('Residual Plot')
plt.grid(True)
plt.show()
```





In []: #13. Interpret results.

```
print("Interpretation:")
print(f"1. The correlation coefficient is {correlation:.3f}, indicating a "
      f"'strong' if abs(correlation)>0.7 else 'moderate' "
      f"'negative' if correlation<0 else 'positive' relationship between horsepower and mpg.")
print(f"2. The regression equation is: mpg = {intercept:.3f} + ({slope:.3f}) * horsepower")
print(f"3. R² = {R2:.3f} means {R2*100:.1f}% of variance in mpg is explained by horsepower.")
print(f"4. As horsepower increases, mpg tends to {'decrease' if slope<0 else 'increase'}.")
```

Interpretation:

1. The correlation coefficient is -0.778, indicating a strong negative relationship between horsepower and mpg.
2. The regression equation is: $\text{mpg} = 39.936 + (-0.158) * \text{horsepower}$
3. $R^2 = 0.606$ means 60.6% of variance in mpg is explained by horsepower.
4. As horsepower increases, mpg tends to decrease.